

آموزش گیت و گیت‌هاب

علیرضا فرهی قهی

نیم‌سال دوم سال تحصیلی ۱۴۰۴-۱۴۰۵

فهرست مطالب

۱	نصب گیت	۱
۱	پیکربندی (configure) گیت	۲
۱	راه‌اندازی اولیه (Initialize)	۳
۲	حذف مخزن گیت	۴
۲	انواع فایل از نظر گیت	۵
۳	دستورهای اولیه	۶
۳	۱.۶ Status (وضعیت)	۱.۶
۳	۲.۶ List Files (فهرست فایل‌های ردیابی شده)	۲.۶
۳	توضیح کاربرد برای فایل‌های Unmodified	۳
۳	۳.۶ Stage (آماده‌سازی)	۳.۶
۴	۴.۶ Untrack (خارج کردن فایل از حالت ردیابی شده)	۴.۶
۴	۵.۶ Unstage (خارج کردن فایل از حالت سکو)	۵.۶
۴	۶.۶ Commit (ثبت تغییرات)	۶.۶
۵	اضافه کردن توضیحات بیشتر به کامیت	۵
۵	میان‌بر: Commit و Add به‌طور همزمان	۵
۵	۷.۶ Log (مشاهده تاریخچه)	۷.۶
۶	مشاهده کامیت‌ها با فرمت‌های مختلف	۶
۶	۸.۶ Amend (ویرایش آخرین commit)	۸.۶

۶	تغییر پیام آخرین commit	۶
۶	اضافه کردن فایل های فراموش شده	۶
۷	Diff (مشاهده تغییرات)	۹.۶
۷	Checkout (جابه جایی بین شاخه ها و کامیت ها)	۱۰.۶
۷	جابه جایی بین شاخه ها	۷
۸	مشاهده کامیت های قدیمی	۸
۸	بازگشت به آخرین commit ثبت شده	۸
۸	Reset (لغو و پاک کردن کامیت ها)	۱۱.۶
۸	حالت Soft (ملایم)	۸
۹	حالت Mixed (متوسط / پیش فرض)	۹
۹	حالت Hard (خشن و خطرناک)	۹
۹	Branching & Merging (مدیریت شاخه ها)	۱۲.۶
۹	مشاهده، ساخت و حذف شاخه ها (Branch)	۹
۱۰	ادغام شاخه ها (Merge)	۱۰
۱۰	Revert (لغو ایمن تغییرات)	۱۳.۶
۱۱	Stash (ذخیره موقت تغییرات)	۱۴.۶
۱۱	ذخیره تغییرات در حافظه موقت	۱۱
۱۱	مشاهده لیست تغییرات ذخیره شده	۱۱
۱۱	بازگرداندن تغییرات و ادامه کار	۱۱
۱۲	بازیابی یک Stash خاص از میان چندین مورد	۱۲
۱۲	ارتباط با مخازن راه دور (Remote & GitHub)	۷
۱۲	ایجاد حساب در گیت هاب	۱۰.۷
۱۲	پیکربندی کلید SSH (اتصال امن و بدون رمز به گیت هاب)	۲۰.۷
۱۲	مرحله اول: ساخت کلید SSH	۱۲
۱۲	مرحله دوم: کپی کردن کلید عمومی	۱۲
۱۳	مرحله سوم: ثبت کلید در سایت گیت هاب	۱۳
۱۳	مرحله چهارم: تست اتصال	۱۳
۱۳	اتصال پروژه محلی فعلی به گیت هاب (Remote)	۳.۷
۱۴	ارسال تغییرات به سرور (Push)	۴.۷
۱۴	دریافت تغییرات از سرور (Pull)	۵.۷
۱۴	دریافت یک پروژه از اینترنت (Clone)	۶.۷
۱۵	کار تیمی در گیت هاب (روش افزودن همکار یا Collaborator)	۸

۱۵	۹ نادیده گرفتن فایل‌های اضافی (.gitignore)
۱۶	۱.۹ فایل .gitignore چگونه ساخته می‌شود؟
۱۷	۱۰ فایل README.md (ویترین پروژه شما)
۱۷	۱.۱۰ فرمت مارک‌داون (Markdown)
۱۷	۲.۱۰ چند دستور پرکاربرد در مارک‌داون

۱ نصب گیت

در ویندوز، برای نصب گیت در ترمینال ویندوز (چه Command Prompt و چه PowerShell) دستور زیر را وارد می‌کنیم:

```
winget install Git.Git
```

و در لینوکس، دستورات زیر را وارد می‌کنیم:

```
sudo apt update  
sudo apt install git
```

۲ پیکربندی (configure) گیت

برای پیکربندی سراسری گیت، از دستورات زیر استفاده می‌کنیم:

```
git config --global user.name "<your name>"  
git config --global user.email "<your email>"
```

باید توجه داشته باشیم که دستور `git config` به صورت پیش فرض محلی (Local) است و تنها در صورتی که پوشه مدنظر یک مخزن گیت باشد، این دستور به درستی کار می‌کند. همچنین می‌توانیم پس از پیکربندی سراسری، برای مخزن‌های دلخواه، پیکربندی خاصی را به صورت محلی به کمک دستور `git config` اعمال کنیم.

توجه داشته باشید که ایمیل شما حتما باید با ایمیل حساب گیت‌هاب شما یکسان باشد (تا گیت‌هاب بتواند تغییرات را به حساب شما متصل کند)، اما نام شما می‌تواند نام دلخواه باشد و لزومی ندارد دقیقا مشابه نام کاربری گیت‌هاب شما باشد.

به کمک دستور زیر، می‌توانیم پیکربندی‌های موجود و همچنین مسیر فایلی که هر پیکربندی در آن ذخیره شده است را مشاهده کنیم:

```
git config --list --show-origin
```

۳ راه‌اندازی اولیه (Initialize)

برای تبدیل یک پوشه معمولی به یک مخزن گیت، در مسیر آن پوشه دستور زیر را وارد می‌کنیم. با اجرای این دستور، یک پوشه (Directory) پنهان به نام `.git` در آن مسیر ساخته می‌شود. این پوشه، قلب تپنده مخزن شماست و گیت تمام تاریخچه و تنظیمات را در آن ذخیره می‌کند.

```
git init
```

پس از اجرای این دستور، پوشه شما به یک مخزن گیت تبدیل می‌شود. البته توجه داشته باشید که گیت هیچ فایلی را به‌طور خودکار تحت نظر (Track) نمی‌گیرد و برای این کار باید صراحتاً فایل‌ها را به گیت معرفی کنیم.

۴ حذف مخزن گیت

از آنجایی که مخزن گیت تنها یک پوشه پنهان است، برای لغو کردن گیت در یک پوشه (و تبدیل مجدد آن به یک پوشه معمولی) دستور خاصی در خود گیت وجود ندارد و باید پوشه `.git` را به کمک دستورات سیستم‌عامل پاک کنیم. برای حذف این پوشه پنهان از طریق ترمینال، بسته به محیطی که استفاده می‌کنید، از دستورات زیر کمک بگیرید:

در محیط CMD (ویندوز):

```
rmdir /s /q .git
```

در محیط PowerShell (ویندوز):

```
Remove-Item -Force -Recurse -Path .git
```

در محیط لینوکس، مک و Git Bash:

```
rm -rf .git
```

۵ انواع فایل از نظر گیت

فایل‌ها در یک مخزن گیت در دو حالت کلی قرار دارند:

۱. **Untracked** (ردگیری نشده): فایل‌های جدیدی که در پوشه ساخته شده‌اند، اما گیت هنوز آن‌ها را نمی‌شناسد و تغییراتشان را دنبال نمی‌کند.

۲. **Tracked** (ردگیری شده): فایل‌هایی که قبلاً به گیت معرفی شده‌اند. این فایل‌ها خود به سه دسته تقسیم می‌شوند:

• **Unmodified** (بدون تغییر): فایل‌هایی که نسبت به آخرین باری که ذخیره (commit) شده‌اند، هیچ تغییری نکرده‌اند.

- **Modified** (تغییر یافته): فایل‌هایی که قبلاً در گیت ذخیره شده‌اند، اما شما کدهای آن‌ها را ویرایش کرده‌اید و هنوز این تغییرات جدید را برای ثبت نهایی آماده نکرده‌اید.
- **Staged** (آماده ثبت): فایل‌های جدید یا ویرایش‌شده‌ای که آن‌ها را به اصطلاح روی سکو قرار داده‌ایم و آماده هستند تا در مرحله بعدی (commit)، تغییراتشان در تاریخچه گیت ثبت شود.

۶ دستوره‌های اولیه

۱.۶ Status (وضعیت)

برای مشاهده فایل‌های Modified، Staged و Untracked در مخزن، از دستور زیر استفاده می‌کنیم:

```
git status
```

۲.۶ List Files (فهرست فایل‌های ردیابی شده)

برای مشاهده فایل‌های Tracked در مخزن، از دستور زیر استفاده می‌کنیم:

```
git ls-files
```

توضیح کاربرد برای فایل‌های Unmodified

فایل‌هایی که در وضعیت Unmodified هستند، در خروجی git status نمایش داده نمی‌شوند. برای دیدن فایل‌هایی که در وضعیت Unmodified هستند، باید لیست تمام فایل‌های Tracked توسط گیت را (به کمک دستور git ls-files) بگیریم و آن‌هایی که در وضعیت‌های دیگر (Modified یا Staged که به کمک دستور git status می‌توان آن‌ها را یافت) قرار دارند را فیلتر کنیم.

۳.۶ Stage (آماده‌سازی)

برای انتقال یک فایل به حالت Staged، از دستور زیر استفاده می‌کنیم:

```
git add <path to the file>
```

هشدار: اگر نام فایل یا پوشه شما دارای فاصله (Space) یا کاراکترهای خاص (مثل &) است، باید کل نام فایل را داخل علامت نقل قول (کوته‌نویس) قرار دهید تا ترمینال دچار خطا نشود. مثال:

```
git add "Learn Git & GitHub.tex"
```

نکته: برای اینکه وضعیت تمام فایل‌های تغییر یافته و جدید درون پوشه را به صورت یکجا به حالت Staged تغییر دهیم، از پرچم -A (یا نقطه .) استفاده می‌کنیم:

```
git add -A
```

۴.۶ Untrack (خارج کردن فایل از حالت ردیابی شده)

برای خارج کردن فایل‌ها از حالت Tracked، از این دستور استفاده می‌شود:

```
git rm --cached <path to files>
```

نکته: اگر بخواهیم تمام فایل‌ها و پوشه‌های مخزن را یکجا از حالت Tracked خارج کنیم، استفاده از نقطه (.) به تنهایی خطا می‌دهد و باید از پرچم -r (برای اعمال روی پوشه‌ها) استفاده کنیم:

```
git rm -r --cached .
```

۵.۶ Unstage (خارج کردن فایل از حالت سکو)

برای خارج کردن فایل‌ها از حالت Staged و برگرداندن آن‌ها به حالت قبلی (Untracked یا Modified)، از دستور زیر استفاده می‌کنیم:

```
git restore --staged <path to file>
```

هشدار (خطای HEAD): دستور git restore تنها در صورتی کار می‌کند که مخزن شما حداقل یک کامیت ثبت شده داشته باشد تا بتواند به آن برگردد. اگر در یک مخزن کاملاً جدید هستید که هنوز هیچ کامیتی در آن ثبت نشده است، این دستور با خطای

```
fatal: could not resolve 'HEAD'
```

مواجه می‌شود و باید از دستور git rm --cached استفاده کنید.

۶.۶ Commit (ثبت تغییرات)

مهم‌ترین بخش کار با گیت، ثبت نهایی تغییرات است. Commit مانند گرفتن یک عکس یادگاری از وضعیت فعلی پروژه است. گیت تنها تغییرات فایل‌هایی را در Commit ذخیره می‌کند که در حالت Staged قرار داشته باشند.

برای ثبت تغییرات به همراه یک پیام کوتاه، از دستور زیر استفاده می‌کنیم:

```
git commit -m "<Commit message>"
```

هشدار (تله ادیتور Vim): اگر در دستور خود پرچم -m را فراموش کنید و فقط git commit را وارد کنید، گیت شما را وارد ویرایشگر متنی ترمینال (معمولا Vim) می‌کند که کار با آن برای مبتدیان کمی گیج‌کننده است. در صورت ورود اشتباه به این محیط، دکمه Esc را بزنید، عبارت !q: را تایپ کرده و Enter بزنید تا خارج شوید.

اضافه کردن توضیحات بیشتر به کامیت

اگر تغییرات شما زیاد است و نیاز دارید علاوه بر عنوان Commit، توضیحات مفصل‌تری هم درباره کارهایی که انجام داده‌اید بنویسید، می‌توانید از دو پرچم -m به شکل زیر استفاده کنید (پرچم اول به عنوان Subject و پرچم دوم به عنوان Body در نظر گرفته می‌شود):

```
git commit -m "<Title>" -m "<Detailed description>"
```

میان‌بر: Commit و Add به‌طور همزمان

اگر فایل‌های شما از قبل توسط گیت Tracked شده‌اند و فقط کدهای آن‌ها را ویرایش کرده‌اید، می‌توانید برای سرعت بیشتر مرحله git add را دور بزنید و فایل‌ها را با پرچم -am (یا -a -m) commit کنید:

```
git commit -am "<Commit message>"
```

توجه کنید که این میان‌بر روی فایل‌های Untracked کار نمی‌کند.

۷.۶ Log (مشاهده تاریخچه)

برای دیدن اطلاعات مربوط به کامیت‌های قبلی می‌توانید از دستور زیر استفاده کنید:

```
git log
```

هر commit شامل اطلاعات زیر است:

• Commit Hash

• Author

• Timestamp

• Commit Message

مشاهده کامیت‌ها با فرمت‌های مختلف

می‌توان با استفاده از پرچم‌ها، کامیت‌های قبلی را در فرمت‌های مختلفی مشاهده کرد. برای مثال، دستور ترکیبی زیر تاریخچه کامیت‌ها را به شکل یک گراف نمایش می‌دهد:

```
git log --all --decorate --oneline --graph
```

نکته: این دستور به‌ویژه زمانی که در پروژه خود چندین شاخه (Branch) مختلف داشته باشید، برای درک مسیر توسعه پروژه بسیار راهگشا خواهد بود.

۸.۶ Amend (ویرایش آخرین commit)

هنگامی که فراموش کردید فایلی را در commit اضافه کنید یا خواستید توضیحات مربوط به آخرین commit را تغییر دهید، می‌توانید از این قابلیت استفاده کنید. با انجام عملیات amend، گیت در واقع commit قبلی را پاک کرده و یک commit جدید می‌سازد؛ بنابراین هش (Hash) کامیت مورد نظر تغییر خواهد کرد.

تغییر پیام آخرین commit

□ روش اول: با وارد کردن دستور زیر، یک ویرایشگر متنی (Editor) در ترمینال باز می‌شود که می‌توانید توضیحات commit قبلی را در آن تغییر دهید و سپس فایل را ذخیره کنید تا commit اصلاح شود:

```
git commit --amend
```

□ روش دوم: اگر نمی‌خواهید درگیر ویرایشگرهای متنی شوید، می‌توانید مستقیماً پیام جدید را با پرچم -m وارد کنید:

```
git commit --amend -m "<New commit message>"
```

اضافه کردن فایل‌های فراموش شده

اگر فایلی را از قلم انداخته‌اید، ابتدا باید فایل‌های خود را با دستور git add در وضعیت staged قرار دهید. سپس یکی از دستورات زیر را وارد کنید:

□ حالت اول (تغییر فایل + تغییر پیام): اگر علاوه بر اضافه کردن فایل، می‌خواهید پیام commit را هم تغییر دهید، کافی است دستور git commit --amend را بزنید تا ویرایشگر باز شود.

□ حالت دوم (فقط اضافه کردن فایل بدون تغییر پیام): اگر می‌خواهید پیام کامیت قبلی دقیقاً همان‌طور که بود دست‌نخورده باقی بماند و فقط فایل‌ها را به آن سنجاق شود، از پرچم --no-edit استفاده کنید:

```
git add <forgotten_file>
git commit --amend --no-edit
```

هشدار: از آنجایی که با این کار هش کامیت تغییر می‌کند، هرگز کامیت‌هایی که قبلاً به یک مخزن راه دور (مثل گیت‌هاب) ارسال (Push) شده‌اند را amend نکنید، زیرا باعث به هم ریختن تاریخچه پروژه برای سایر هم‌تیمی‌ها می‌شود.

۹.۶ Diff (مشاهده تغییرات)

برای دیدن تغییرات فایل‌های commit شده نسبت به آخرین commit، می‌توانید از این دستور استفاده کنید:

```
git diff
```

برای مشاهده تغییرات دو commit دلخواه نسبت به هم، کافی است که هش یا آدرس HEAD-based کامیت‌های مورد نظر را به ترتیب وارد کنید:

```
git diff <src commit> <dst commit>
```

۱۰.۶ Checkout (جابه‌جایی بین شاخه‌ها و کامیت‌ها)

دستور checkout یکی از پرکاربردترین دستورات گیت است که برای جابه‌جایی بین شاخه‌ها (Branches) و همچنین بازگردانی وضعیت فایل‌ها به کامیت‌های گذشته استفاده می‌شود. این دستور به گیت می‌گوید که اشاره‌گر HEAD را به نقطه دیگری منتقل کند.

جابه‌جایی بین شاخه‌ها

برای رفتن از یک شاخه به شاخه دیگر، نام آن شاخه را مقابل دستور می‌نویسیم:

```
git checkout <branch_name>
```

نکته (میان‌بر ساخت شاخه): برای ایجاد یک شاخه جدید و ورود همزمان به آن، از پرچم -b استفاده می‌شود:

```
git checkout -b <new_branch_name>
```

مشاهده کامیت‌های قدیمی

اگر نیاز دارید به کامیت‌های قبلی برگردید، دستور زیر را وارد کنید. در این دستور لازم است که هش یا آدرس commit HEAD-based مورد نظر را وارد کنید:

```
git checkout <commit>
```

نکته (بازگشت سریع): برای برگرداندن HEAD به commit قبلی، می‌توانید از دستور زیر نیز استفاده کنید:

```
git checkout -
```

این دستور شما را به آخرین شاخه یا کامیتی که پیش از این در آن قرار داشتید، برمی‌گرداند.

بازگشت به آخرین commit ثبت‌شده

زمانی که با وارد کردن هش یک کامیت به گذشته سفر می‌کنید، گیت شما را در وضعیت موقتی به نام Detached قرار می‌دهد. برای خروج از این حالت و بازگشت مجدد به آخرین کامیت ثبت‌شده در شاخه اصلی، نیازی به وارد کردن هش نیست و تنها کافی است نام شاخه اصلی خود (معمولا master یا main) را صدا بزنید:

```
git checkout master
```

نکته: می‌توان به جای این دستور، از git switch master استفاده کرد.

۱۱.۶ Reset (لغو و پاک کردن کامیت‌ها)

دستور reset برای بازگرداندن پروژه به یک commit در گذشته و پاک کردن تاریخچه بعد از آن استفاده می‌شود. برخلاف checkout که تنها برای مشاهده گذشته است، reset تاریخچه را تغییر داده و کامیت‌های ثبت‌شده را لغو می‌کند.

این دستور دارای سه حالت اصلی است که مشخص می‌کنند چه اتفاقی برای فایل‌های ویرایش‌شده می‌افتد:

حالت Soft (ملایم)

در این حالت، کامیت‌های شما لغو می‌شوند اما تمام تغییراتی که در فایل‌ها داده بودید، به صورت دست‌نخورده در حالت Staged باقی می‌مانند:

```
git reset --soft <commit_hash>
```

حالت Mixed (متوسط / پیش فرض)

اگر هیچ پرچمی وارد نکنید، گیت از این حالت استفاده می‌کند. در این حالت کامیت‌ها لغو شده و فایل‌ها از حالت Staged خارج می‌شوند، اما کدهای شما روی سیستم پاک نمی‌شوند و می‌توانید دوباره آن‌ها را ویرایش کنید:

```
git reset <commit_hash>
```

حالت Hard (خشن و خطرناک)

این حالت commit را لغو کرده و تمامی تغییرات و کدهای شما را برای همیشه از روی سیستم پاک می‌کند تا فایل‌ها دقیقا مشابه وضعیت آن commit قدیمی شوند. هنگام استفاده از این دستور بسیار محتاط باشید:

```
git reset --hard <commit_hash>
```

نکته (میان‌بر HEAD): اگر می‌خواهید فقط یک commit به عقب برگردید و نیازی به پیدا کردن هش commit قبلی نداشته باشید، می‌توانید از عبارت HEAD~1 استفاده کنید (عدد ۱ نشان‌دهنده تعداد کامیت‌هایی است که می‌خواهید به عقب برگردید). مثال:

```
git reset --hard HEAD~1
```

هشدار: مشابه دستور amend، هرگز نباید کامیت‌هایی که قبلا به سرور (مانند گیت‌هاب) Push شده‌اند را reset کنید، زیرا باعث نابودی تاریخچه و ایجاد تداخل شدید برای سایر اعضای تیم می‌شود.

۱۲.۶ Branching & Merging (مدیریت شاخه‌ها)

شاخه‌ها (Branches) در گیت به شما اجازه می‌دهند که یک مسیر توسعه موازی و کاملا ایزوله بسازید. با این کار می‌توانید بدون نگرانی از خراب شدن کدهای اصلی پروژه (master یا main)، روی ویژگی‌های جدید کار کنید یا کدهای آزمایشی بنویسید.

مشاهده، ساخت و حذف شاخه‌ها (Branch)

□ **مشاهده شاخه‌ها:** برای دیدن لیست تمام شاخه‌های موجود از دستور زیر استفاده کنید. شاخه‌ای که در حال حاضر درون آن هستید با رنگ سبز و علامت ستاره مشخص می‌شود:

```
git branch
```

□ ساخت شاخه جدید: برای ایجاد یک شاخه جدید از وضعیت فعلی کدها، نام دلخواه خود را مقابل دستور بنویسید (توجه کنید که این دستور فقط شاخه را می‌سازد و شما را وارد آن نمی‌کند):

```
git branch <branch_name>
```

یادآوری: همان‌طور که در بخش Checkout اشاره شد، برای ساخت و ورود همزمان به یک شاخه می‌توانید از دستور `git checkout -b <branch_name>` استفاده کنید.

□ حذف شاخه: زمانی که کارتان با یک شاخه فرعی تمام شد و آن را با موفقیت ادغام کردید، برای تمیز ماندن محیط کار می‌توانید آن را حذف کنید:

```
git branch -d <branch_name>
```

ادغام شاخه‌ها (Merge)

وقتی کار کدنویسی در یک شاخه فرعی تمام شد و از صحت اجرای آن‌ها مطمئن شدید، باید آن تغییرات را به تنه اصلی پروژه اضافه کنید. به این فرآیند «ادغام» یا Merge می‌گویند. برای ادغام کردن دو شاخه، ابتدا باید وارد شاخه‌ای شوید که می‌خواهید کدها را دریافت کند (یعنی شاخه مقصد که معمولاً master است). پس ابتدا با دستور زیر به شاخه اصلی برمی‌گردیم:

```
git switch master
```

سپس نام شاخه‌ای که تغییرات جدید را در خود دارد صدا می‌زنیم تا محتویاتش با شاخه فعلی ترکیب شود:

```
git merge <branch_name>
```

پس از اجرای این دستور، گیت تلاش می‌کند تغییرات را به صورت خودکار ترکیب کند. اگر دو نفر همزمان روی یک خط از یک فایل تغییراتی ایجاد کرده باشند، گیت متوقف شده و به شما هشدار تداخل (Conflict) می‌دهد تا خودتان به صورت دستی تصمیم بگیرید کدام کد نهایی شود.

۱۳.۶ Revert (لغو ایمن تغییرات)

برخلاف دستور reset که تاریخچه کامیت‌ها را پاک می‌کند و استفاده از آن برای کارهایی که روی سرور قرار گرفته‌اند خطرناک است، دستور revert یک روش ایمن برای لغو تغییرات است.

این دستور commit قبلی را پاک نمی‌کند، بلکه یک کامیت جدید می‌سازد که دقیقاً تغییرات commit مورد نظر شما را خنثی می‌کند. بنابراین تاریخچه پروژه دست‌نخورده باقی می‌ماند و هم‌تیمی‌های شما دچار مشکل نمی‌شوند.

برای خنثی کردن یک commit خاص، هش (Hash) آن را به دستور زیر بدهید:

```
git revert <commit_hash>
```

پس از اجرای این دستور، یک ویرایشگر متنی باز می‌شود تا پیام commit جدید (که عمل لغو را انجام داده) را تایید یا ویرایش کنید.

۱۴.۶ Stash (ذخیره موقت تغییرات)

گاهی در حال انجام تغییرات روی یک فایل هستید که مجبور می‌شوید به شاخه دیگری بروید (مثلاً برای رفع یک خطای فوری). از آنجایی که کدهایتان هنوز ناقص است، نمی‌خواهید آن‌ها را commit کنید. در این شرایط از قابلیت stash استفاده می‌کنیم. این دستور کدهای تغییر یافته شما را در یک فضای موقت ذخیره می‌کند تا محیط کارتان کاملاً تمیز شود.

ذخیره تغییرات در حافظه موقت

```
git stash
```

مشاهده لیست تغییرات ذخیره شده

اگر چندین بار از این قابلیت استفاده کرده‌اید، برای دیدن لیست آن‌ها دستور زیر را وارد کنید:

```
git stash list
```

بازگرداندن تغییرات و ادامه کار

پس از اینکه کارتان در شاخه دیگر تمام شد و به شاخه اصلی خود برگشتید، با دستور زیر کدهای نصفه نیمه خود را بازگردانید (این دستور تغییرات را روی فایل‌ها اعمال کرده و سپس آن را از حافظه موقت پاک می‌کند):

```
git stash pop
```

نکته: دستور pop تغییرات را برگردانده و از حافظه موقت پاک می‌کند. اگر می‌خواهید تغییرات برگردند اما همچنان در حافظه stash باقی بمانند (مثلاً برای اعمال روی یک شاخه دیگر)، به جای pop از دستور apply استفاده کنید:

```
git stash apply
```

بازیابی یک Stash خاص از میان چندین مورد

زمانی که چندین بار کدهای خود را stash می‌کنید، گیت آن‌ها را شماره‌گذاری می‌کند. ابتدا با دستور `git stash list` شماره (Index) مورد نظر را پیدا کنید (مثلا `stash@{1}`). سپس با اضافه کردن آن شماره به انتهای دستور `pop`، همان کشوی خاص را بازگردانید:

```
git stash pop stash@{1}
```

۷ ارتباط با مخازن راه دور (Remote & GitHub)

۱.۷ ایجاد حساب در گیت‌هاب

اگر حساب GitHub ندارید، می‌توانید با مراجعه به github.com یک حساب بسازید.

۲.۷ پیکربندی کلید SSH (اتصال امن و بدون رمز به گیت‌هاب)

برای اتصال سیستم خود به گیت‌هاب بدون نیاز به وارد کردن مداوم نام کاربری و رمز عبور، بهترین و استانداردترین روش استفاده از کلیدهای رمزنگاری شده SSH است. این فرایند شامل ساخت یک جفت کلید (عمومی و خصوصی) روی کامپیوتر شما و قرار دادن کلید عمومی در سایت گیت‌هاب است.

مرحله اول: ساخت کلید SSH

ترمینال خود (در ویندوز Git Bash) را باز کرده و دستور زیر را وارد کنید. (دقت کنید که به جای ایمیل فرضی، ایمیلی که با آن در گیت‌هاب ثبت‌نام کرده‌اید را درون کوتیشن بنویسید):

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

پس از وارد کردن این دستور، سیستم از شما مسیر ذخیره‌سازی و رمز عبور اضافه (Passphrase) می‌خواهد. برای راحتی کار و تنظیمات پیش‌فرض، فقط سه بار کلید `Enter` را فشار دهید تا کلیدها ساخته شوند.

مرحله دوم: کپی کردن کلید عمومی

حالا باید محتویات کلید عمومی (Public Key) ساخته شده را کپی کنید. برای خواندن این فایل، اگر از Git Bash (یا سیستم‌عامل‌های لینوکس و مک) استفاده می‌کنید، دستور زیر را وارد کنید:

```
cat ~/.ssh/id_ed25519.pub
```

نکته (برای کاربران ویندوز): اگر از خط فرمان پیش فرض ویندوز (CMD) استفاده می‌کنید، دستور cat کار نخواهد کرد و باید از معادل ویندوزی آن استفاده کنید:

```
type "%USERPROFILE%\.ssh\id_ed25519.pub"
```

پس از اجرای دستور، تمام متن چاپ شده در صفحه (که معمولاً با عبارت ssh-ed25519 شروع می‌شود) را کپی کنید.

مرحله سوم: ثبت کلید در سایت گیت‌هاب

۱. وارد سایت GitHub شوید و روی عکس پروفایل خود کلیک کنید و به بخش Settings بروید.
۲. از منوی سمت چپ، گزینه SSH and GPG keys را انتخاب کنید.
۳. روی دکمه سبز رنگ New SSH key کلیک کنید.
۴. در بخش Title یک عنوان دلخواه (مثلاً Asus Laptop) بنویسید.
۵. متنی که در مرحله قبل کپی کرده بودید را در کادر بزرگ Paste Key کنید و در نهایت دکمه Add SSH key را بزنید.

مرحله چهارم: تست اتصال

برای اطمینان از اینکه اتصال لپ‌تاپ شما با سرورهای گیت‌هاب با موفقیت برقرار شده است، دستور زیر را در ترمینال وارد کنید:

```
ssh -T git@github.com
```

اگر در این مرحله سوالی پرسیده شد، کلمه yes را تایپ کرده و اینتر بزنید. اگر پیام موفقیتی شامل نام کاربری خود دریافت کردید، یعنی کارت VIP شما فعال شده است!

۳.۷ اتصال پروژه محلی فعلی به گیت‌هاب (Remote)

گاهی اوقات شما از قبل پروژه‌ای را روی سیستم خود ساخته‌اید و حالا می‌خواهید آن را به یک مخزن جدید در گیت‌هاب متصل کنید. در ادبیات گیت، به مخزن روی سرور، Remote و به آدرس پیش فرض آن، origin می‌گویند.

برای اتصال پروژه محلی به گیت‌هاب از دستور زیر استفاده می‌کنیم:

```
git remote add origin <Repository_SSH_URL>
```


برای بررسی اینکه پروژه شما به چه آدرسی و با چه پروتکلی متصل است، می‌توانید از پرچم `-v` استفاده کنید:

```
git remote -v
```

۴.۷ ارسال تغییرات به سرور (Push)

پس از ثبت تغییرات، این تغییرات تنها روی کامپیوتر شما وجود دارند. برای ارسال و آپلود آن‌ها روی سرور گیت‌هاب، از دستور `git push` استفاده می‌کنیم. برای اولین بار که می‌خواهید یک شاخه (مانند `master` یا `main`) را به سرور ارسال کنید، باید بین شاخه محلی و شاخه روی سرور یک پیوند (Tracking) ایجاد کنید. این کار با پرچم `-u` انجام می‌شود:

```
git push -u origin master
```

نکته: پس از اجرای این دستور در اولین بار، گیت مسیر این شاخه را به خاطر می‌سپارد و برای ارسال کامیت‌های بعدی در همین شاخه، نیازی به وارد کردن نام شاخه نیست و تنها نوشتن دستور زیر کفایت می‌کند:

```
git push
```

۵.۷ دریافت تغییرات از سرور (Pull)

منظور از `pull` کردن تغییرات، دانلود کردن تغییرات و کامیت‌ها از مخزن GitHub است. به عبارت دیگر، زمانی که کدهای روی سرور به‌روزتر از کدهای روی سیستم شما باشند (مثلاً هم‌تیمی شما تغییراتی را ارسال کرده باشد)، با این کار پروژه محلی خود را با آخرین وضعیت سرور همگام‌سازی می‌کنید. برای دریافت این تغییرات، وارد پوشه پروژه شده و از دستور زیر استفاده کنید:

```
git pull
```

۶.۷ دریافت یک پروژه از اینترنت (Clone)

دستور `clone` برای کپی کردن کامل یک مخزن از گیت‌هاب بر روی کامپیوتر شخصی استفاده می‌شود. با اجرای این دستور، گیت یک پوشه جدید با نام مخزن می‌سازد و تمام فایل‌ها، تاریخچه کامیت‌ها و شاخه‌ها را درون آن دانلود می‌کند.

برای `clone` کردن یک مخزن، کافی است وارد پوشه دلخواه خود در ترمینال شوید و از دستور زیر استفاده کنید:

```
git clone <Repository_SSH_URL>
```

نکته: از آنجایی که در مراحل قبل کلید SSH را تنظیم کردیم، حتما هنگام کپی کردن لینک مخزن از سایت گیت‌هاب، تب SSH (لینکی که با `git@github.com` شروع می‌شود) را انتخاب کنید.

۸ کار تیمی در گیت‌هاب (روش افزودن همکار یا Collaborator)

زمانی که قصد دارید با یک یا چند نفر از هم‌تیمی‌های خود روی یک پروژه مشترک کار کنید، ساده‌ترین و رایج‌ترین روش، اضافه کردن آن‌ها به عنوان همکار (Collaborator) در مخزن گیت‌هاب است. در این روش، تمام اعضای تیم دسترسی مستقیم برای Pull و Push کدها را خواهند داشت.

مراحل ایجاد دسترسی تیمی:

۱. ساخت و راه‌اندازی مخزن: ابتدا یک نفر به عنوان سازنده پروژه، مخزن را در اکانت گیت‌هاب خود ایجاد کرده و کدهای اولیه را در آن Push می‌کند.

۲. ارسال دعوت‌نامه به هم‌تیمی‌ها: سازنده پروژه در سایت گیت‌هاب وارد صفحه مخزن شده و به Settings می‌رود. سپس از منوی کناری، بخش Collaborators را انتخاب کرده و با کلیک روی دکمه Add people، نام کاربری (Username) یا ایمیل هم‌تیمی‌های خود را جستجو و اضافه می‌کند.

۳. پذیرش دعوت و Clone کردن: هم‌تیمی‌ها یک ایمیل دعوت از گیت‌هاب دریافت می‌کنند. پس از کلیک روی لینک تایید در ایمیل، آن‌ها به پروژه اضافه می‌شوند. حالا هر هم‌تیمی باید لینک SSH مخزن را کپی کرده و در سیستم خود دستور زیر را اجرا کند تا پروژه دانلود شود:

```
git clone <Repository_SSH_URL>
```

۴. چرخه کار روزانه: از این پس، هر یک از اعضای تیم برای جلوگیری از تداخل کدها، باید چرخه زیر را رعایت کنند:

- در ابتدای روز: `git pull` (برای دریافت آخرین کدهای نوشته شده توسط دیگران)
- انجام تغییرات و ثبت آن‌ها: `git add` و `git commit`
- در انتهای روز: `git push` (برای ارسال تغییرات جدید به سرور)

۹ نادیده گرفتن فایل‌های اضافی (.gitignore)

در هر پروژه برنامه‌نویسی، ابزارها و سیستم‌عامل‌ها یک سری فایل موقت و سنگین تولید می‌کنند (مثل پوشه `node_modules` در جاوا اسکریپت، فایل‌های کامپایل شده `.exe` یا فایل‌های کش). قرار دادن این فایل‌ها در گیت باعث کندی، افزایش حجم مخزن و ایجاد تداخل در کار تیمی می‌شود.

قانون طلایی گیت این است: ما تنها سورس‌کدهایی که خودمان نوشته‌ایم را در گیت ذخیره می‌کنیم.

۱.۹ فایل gitignore چگونه ساخته می‌شود؟

این فایل تنها یک فایل متنی ساده است. تنها نکته مهم اسم آن است: نام این فایل دقیقاً gitignore است (یعنی هیچ کلمه‌ای قبل از نقطه وجود ندارد و پسوند هم ندارد). برای ایجاد آن سه راه ساده دارید:

۱. ساخت دستی: در ادیتورکد خود (مثل VS Code) در پوشه اصلی پروژه، یک فایل جدید بسازید و نام آن را دقیقاً gitignore بگذارید.

۲. از طریق گیت‌هاب: وقتی در سایت گیت‌هاب در حال ساخت یک مخزن جدید (New Repository) هستید، گزینه‌ای به نام Add gitignore وجود دارد. با انتخاب زبان پروژه‌تان، گیت‌هاب خودش این فایل را با تنظیمات استاندارد می‌سازد.

۳. استفاده از ابزارهای آنلاین: می‌توانید وارد سایت‌هایی مثل [toptal](#) شوید، نام زبان برنامه‌نویسی یا محیط کار خود را وارد کنید و متنی که به شما می‌دهد را درون فایل خود کپی کنید.

نمونه محتویات فایل gitignore

کافیست نام فایل‌ها، پوشه‌ها یا پسوندهایی که می‌خواهید گیت نادیده بگیرد را در این فایل بنویسید (استفاده از ستاره (*) به معنای «هر فایلی با این پسوند» است):

```
# نادیده گرفتن یک پوشه خاص (مثل پکیج‌ها)
node_modules/

# نادیده گرفتن تمام فایل‌ها با یک پسوند خاص (مثل لاگ‌ها یا کش)
*.log
*.pyc
*.exe

# نادیده گرفتن فایل‌های سیستمی ویندوز و مک
.DS_Store
Thumbs.db
```

پس از ذخیره این فایل، اگر دستور `git status` را اجرا کنید، می‌بینید که گیت دیگر این فایل‌های زباله را نمی‌بیند و دستور `git add` کاملاً ایمن خواهد بود.

نکته: آیا خود gitignore هم باید Commit شود؟ بله، فایل gitignore حاوی «قوانین تمیزی پروژه» است و این قوانین باید بین تمام اعضای تیم مشترک باشد. اگر این فایل را Track و Commit نکنید، هم‌تیمی‌های شما آن را دریافت نکرده و ممکن است به اشتباه فایل‌های زباله سیستم خودشان را روی سرور آپلود کنند.

۱۰ فایل README.md (ویترین پروژه شما)

فایل README مهم‌ترین فایل مستندسازی در هر پروژه است و حکم ویترین مخزن شما را در گیت‌هاب دارد. زمانی که شخصی وارد صفحه پروژه شما می‌شود، اولین چیزی که گیت‌هاب به صورت خودکار و در ابعاد بزرگ نمایش می‌دهد، محتویات همین فایل است.

چرا به فایل README نیاز داریم؟

بدون این فایل، پروژه شما تنها یک پوشه حاوی کدهای نامفهوم است. یک فایل README استاندارد باید به این سه سوال اساسی پاسخ دهد:

۱. این پروژه چیست؟ (توضیح مختصری از هدف پروژه و مشکلی که حل می‌کند).
۲. چگونه نصب یا اجرا می‌شود؟ (پیش‌نیازها و دستورات لازم برای اجرای کدها روی سیستم کاربر).
۳. چگونه باید از آن استفاده کرد؟ (ارائه چند مثال ساده یا تصویر از خروجی پروژه).

۱.۱۰ فرمت مارک‌داون (Markdown)

معمولاً این فایل را با نام README.md در ریشه پروژه می‌سازند. پسوند md مخفف Markdown است؛ یک زبان نشانه‌گذاری بسیار سبک و خوانا است. گیت‌هاب کدهای مارک‌داون را به زیبایی رندر کرده و تیتراها، لیست‌ها، لینک‌ها، جداول و تصاویر را به صورت گرافیکی نمایش می‌دهد.

۲.۱۰ چند دستور پرکاربرد در مارک‌داون

پرکاربردترین دستورات این زبان به همراه کاربردشان به صورت زیر است:

```
# Heading 1 (عنوان اصلی و بزرگ)
## Heading 2 (عنوان فرعی)
### Heading 3 (عنوان کوچک‌تر)

**Bold Text** (متن ضخیم یا بولد)
*Italic Text* (متن کج یا ایتالیک)

- Item 1 (لیست نقطه‌ای)
- Item 2

1. First item (لیست شماره‌دار)
2. Second item

*** (خط افقی جداکننده)
```

> *Blockquote* نقل قول

[Click Here] (<https://github.com>) (ساخت لینک قابل کلیک)

``git status`` (کد درون خطی - برای کلمات خاص یا دستورات)

```
```python
بلوک کد چند خطی (با پشتیبانی از رنگ‌بندی زبان‌ها)
def hello():
 print("Hello GitHub!")
```
```

$E = mc^2$ (رندر فرمول‌های ریاضی لاتک به صورت بلوکی)

توجه داشته باشید که در مارک‌داون، برای آن‌که در یک پاراگراف به خط جدید برویم، باید در انتهای خط قبلی بک‌اسلش (\) و یا دو فاصله (Space) بزنیم. اگر بخواهیم یک پاراگراف جدید ایجاد کنیم، کافی است یک خط خالی (Enter زدن دو بار متوالی) بین دو متن بگذاریم.

نکته: در گیت‌هاب، زمانی که وارد مخزن خود می‌شوید، یک تب به نام Preview وجود دارد که به شما اجازه می‌دهد قبل از Commit کردن، خروجی گرافیکی و رندر شده‌ی کدهای مارک‌داون خود را همان‌طور که دیگران می‌بینند، بررسی کنید.